

EXP9: Travelling Salesman Problem (TSP)

Name: Y. Roshitha (192424400)

Aim:

To develop a Python program that solves the Travelling Salesman Problem (TSP) using a brute force approach, so that the shortest possible route visiting every city exactly once and returning to the starting city is determined.

Algorithm:

Step 1: Represent the cities and the distances between them using a cost/distance matrix.

Step 2: Fix the starting city and generate all possible permutations of the remaining cities.

Step 3: For each permutation, calculate the total distance of the route including the return to the starting city.

Step 4: Compare the total distance of the current route with the minimum distance found so far.

Step 5: If the current route is shorter, update the minimum distance and store the route as the best path.

Step 6: Repeat Steps 3 to 5 for all permutations.

Step 7: Display the shortest route and its total distance.

Source Code:

```
# Program to implement Travelling Salesman Problem (TSP) using Brute Force
```

```
from itertools import permutations
```

```
def tsp(graph, start):
    cities = list(graph.keys())
    cities.remove(start)

    min_path = None
    min_cost = float('inf')

    for perm in permutations(cities):
        route = [start] + list(perm) + [start]
        cost = 0
        for i in range(len(route) - 1):
            cost += graph[route[i]][route[i + 1]]

        if cost < min_cost:
            min_cost = cost
            min_path = route

    return min_path, min_cost
```

```
graph = {
```

```

'A': {'A': 0, 'B': 10, 'C': 15, 'D': 20},
'B': {'A': 10, 'B': 0, 'C': 35, 'D': 25},
'C': {'A': 15, 'B': 35, 'C': 0, 'D': 30},
'D': {'A': 20, 'B': 25, 'C': 30, 'D': 0}
}

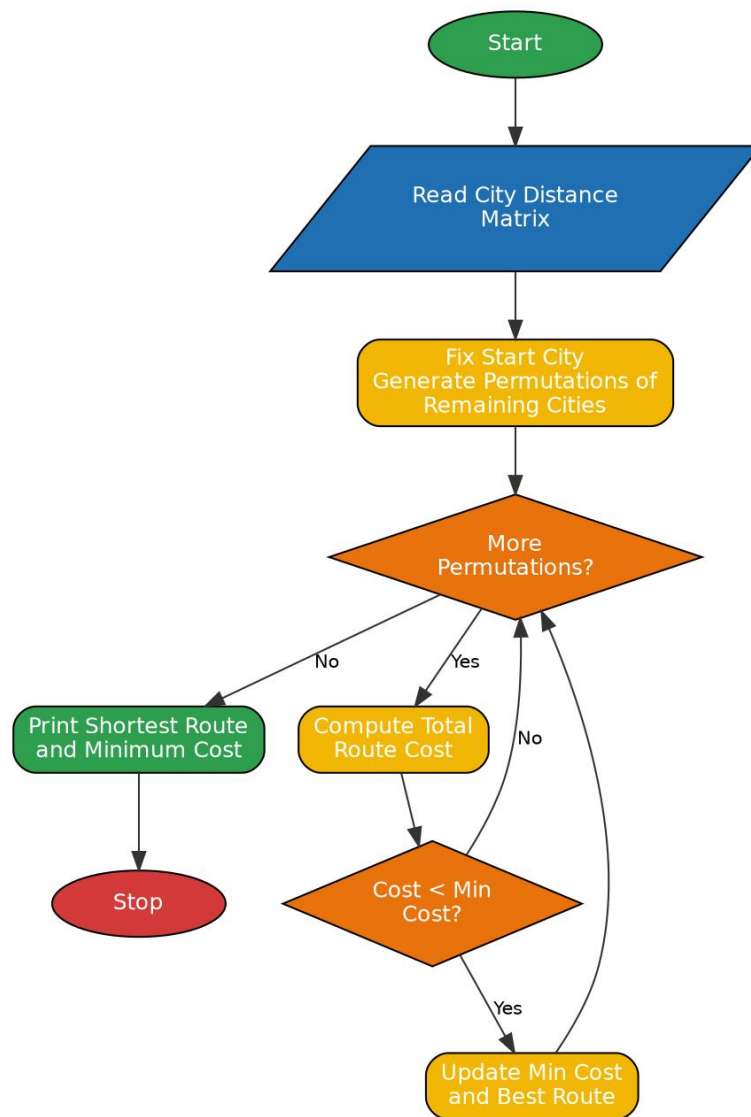
```

```

start_city = 'A'
best_route, best_cost = tsp(graph, start_city)
print("Shortest Route:", " -> ".join(best_route))
print("Minimum Cost :", best_cost)

```

Flowchart:



Output:

```
>>> ===== RESTART: C:/Users/yanar/Documents/CSA1717/EXP9.py =====  
Shortest Route: A -> B -> D -> C -> A  
Minimum Cost   : 80  
>>>
```

Result:

Thus the program to implement the Travelling Salesman Problem (TSP) was written, executed successfully and the output was verified.